

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
	PRACTICAL EXPERIMENT INSTRUCTION SHEET			
	EXPERIMENT TITLE : WRITE A PROGRAM TO IMPLEMENT BEST FIRST SEARCH USING PYTHON			
EXPERIMENT NO. : SSGMCE/WI/IT/6IT09/03		ISSUE NO. : 00	ISSUE DATE : 06-01-2025	
REV. DATE : NA		REV. NO. : 00	DEPTT. :	INFORMATION TECHNOLOGY
LABORATORY : COMPUTER SKILL LAB IV			SEMESTER : VI	PAGE : 1 OF 4

01 AIM : Write A Program To Implement Best First Search using Python

02 SCOPE : The experiment will be focusing on understanding the use of a Best First Search Algorithm as a tool for informed search approach.

03 THEORY :

There are two types of control strategies or search techniques: uninformed and informed. They are explained in detail as given here –

Uninformed Search

It is also called blind search or blind control strategy. It is named so because there is information only about the problem definition, and no other extra information is available about the states. This kind of search techniques would search the whole state space for getting the solution. Breadth First Search (BFS) and Depth First Search (DFS) are the examples of uninformed search.

Informed Search

It is also called heuristic search or heuristic control strategy. It is named so because there is some extra information about the states. This extra information is useful to compute the preference among the child nodes to explore and expand. There would be a heuristic function associated with each node. Best First Search (BFS), A*, Mean and Analysis are the examples of informed search.

Best First Search (Informed Search)

In BFS and DFS, when we are at a node, we can consider any of the adjacent as the next node. So both BFS and DFS blindly explore paths without considering any cost function.

The idea of Best First Search is to use an evaluation function to decide which adjacent is most promising and then explore.

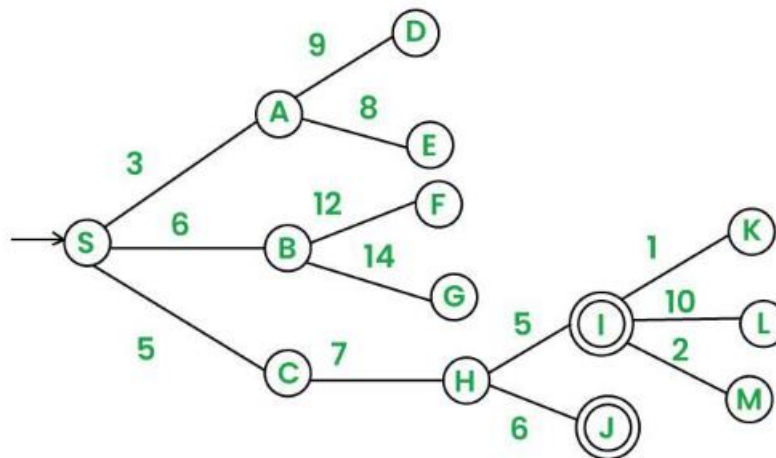
Best First Search falls under the category of Heuristic Search or Informed Search.

Implementation of Best First Search:

We use a priority queue or heap to store the costs of nodes that have the lowest evaluation function value. So the implementation is a variation of BFS, we just need to change Queue to PriorityQueue.

```
// Pseudocode for Best First Search
Best-First-Search(Graph g, Node start)
1) Create an empty PriorityQueue
   PriorityQueue pq;
2) Insert "start" in pq.
   pq.insert(start)
3) Until PriorityQueue is empty
   u = PriorityQueue.DeleteMin
   If u is the goal
     Exit
   Else
     For each neighbor v of u
       If v "Unvisited"
         Mark v "Visited"
         pq.insert(v)
       Mark u "Examined"
End procedure
```

Let us consider the below example:



Weighted Graph

STEPS

We start from source "S" and search for goal "I" using given costs and Best First search.

pq initially contains S

We remove S from pq and process unvisited neighbors of S to pq.

pq now contains {A, C, B} (C is put before B because C has lesser cost)

We remove A from pq and process unvisited neighbors of A to pq.

pq now contains {C, B, E, D}

We remove C from pq and process unvisited neighbors of C to pq.

pq now contains {B, H, E, D}

We remove B from pq and process unvisited neighbors of B to pq.

pq now contains {H, E, D, F, G}

We remove H from pq.

Since our goal "I" is a neighbor of H, we return.

04 PROGRAM

```
from queue import PriorityQueue
v = 14
graph = [[] for i in range(v)]

# Function For Implementing Best First Search
# Gives output path having lowest cost

def best_first_search(actual_Src, target, n):
    visited = [False] * n
    pq = PriorityQueue()
    pq.put((0, actual_Src))
    visited[actual_Src] = True

    while pq.empty() == False:
        u = pq.get()[1]
        # Displaying the path having lowest cost
        print(u, end=" ")
        if u == target:
            break
```

```
    for v, c in graph[u]:
        if visited[v] == False:
            visited[v] = True
            pq.put((c, v))
    print()

# Function for adding edges to graph

def addedge(x, y, cost):
    graph[x].append((y, cost))
    graph[y].append((x, cost))

# The nodes shown in above example(by alphabets) are
# implemented using integers addedge(x,y,cost);
addedge(0, 1, 3)
addedge(0, 2, 6)
addedge(0, 3, 5)
addedge(1, 4, 9)
addedge(1, 5, 8)
addedge(2, 6, 12)
addedge(2, 7, 14)
addedge(3, 8, 7)
addedge(8, 9, 5)
addedge(8, 10, 6)
addedge(9, 11, 1)
addedge(9, 12, 10)
addedge(9, 13, 2)

source = 0
target = 9
best_first_search(source, target, v)
```

05 OUTPUT

0 1 3 2 8 9

06 CONCLUSION

This is how we have successfully implemented the informed search using Best First Search algorithm. The algorithm processes the weights of the neighbors so as to compute the costs and decide the best path.